



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

**Факультет «Информатика и системы управления»
Кафедра «Системы обработки информации и управления»**

**Рубежный контроль №2
по дисциплине «Парадигмы и конструкции языков программирования»**

Выполнил:
студент группы ИУ5-33Б Павлинич А. А.
подпись _____
дата: 20.12.2025

Проверил:
Гапанюк Ю. Е.
подпись _____
дата:

2025 г.

Листинг кода:

```
from typing import List, Optional

class SchoolClass:
    def __init__(self, id: int, name: str):
        self.id = id
        self.name = name

    def __repr__(self):
        return f"SchoolClass(id={self.id}, name='{self.name}')"

class Student:
    def __init__(self, id: int, last_name: str, grade_point_average: float, class_id: int):
        self.id = id
        self.last_name = last_name
        self.grade_point_average = grade_point_average
        self.class_id = class_id

    def __repr__(self):
        return f"Student(id={self.id}, last_name='{self.last_name}', gpa={self.grade_point_average}, class_id={self.class_id})"

class StudentClassRelation:
    def __init__(self, student_id: int, class_id: int):
        self.student_id = student_id
        self.class_id = class_id

    def __repr__(self):
        return f"StudentClassRelation(student_id={self.student_id}, class_id={self.class_id})"

class DataProcessor:
    def __init__(self, classes: List[SchoolClass], students: List[Student],
                 student_relations: List[StudentClassRelation]):
        self.classes = classes
        self.students = students
        self.student_relations = student_relations
```

```

def find_class_by_id(self, class_id: int) -> Optional[SchoolClass]:
    """Вспомогательный метод для поиска класса по ID"""
    return next((cls for cls in self.classes if cls.id == class_id), None)

def find_student_by_id(self, student_id: int) -> Optional[Student]:
    """Вспомогательный метод для поиска студента по ID"""
    return next((s for s in self.students if s.id == student_id), None)

def get_students_starting_with_letter(self, letter: str) -> List[tuple]:
    """Возвращает список студентов с фамилиями на указанную букву и их классы"""
    result = []
    for student in self.students:
        if student.last_name.startswith(letter):
            school_class = self.find_class_by_id(student.class_id)
            if school_class:
                result.append((student.last_name, school_class.name))
    return result

def get_min_gpa_by_class(self) -> List[tuple]:
    """Возвращает список классов с минимальным средним баллом, отсортированный по баллу"""
    class_students = {}
    for student in self.students:
        if student.class_id not in class_students:
            class_students[student.class_id] = []
        class_students[student.class_id].append(student.grade_point_average)

    result = []
    for class_id, gpas in class_students.items():
        school_class = self.find_class_by_id(class_id)
        if school_class:
            min_gpa = min(gpas)
            result.append((school_class.name, min_gpa))

    return sorted(result, key=lambda x: x[1])

def get_all_student_class_relations(self) -> List[tuple]:
    """Возвращает все связи студент-класс, отсортированные по студентам"""
    connections = []
    for rel in self.student_relations:
        student = self.find_student_by_id(rel.student_id)

```

```

        school_class = self.find_class_by_id(rel.class_id)

        if student and school_class:

            connections.append((student.last_name, school_class.name))

    return sorted(connections, key=lambda x: x[0])

def main():

    classes = [

        SchoolClass(1, "5А"),

        SchoolClass(2, "6Б"),

        SchoolClass(3, "7В"),

        SchoolClass(4, "8А"),

        SchoolClass(5, "9Б")

    ]

    students = [

        Student(1, "Андреев", 4.5, 1),

        Student(2, "Петров", 3.8, 2),

        Student(3, "Александрова", 4.7, 1),

        Student(4, "Сидоров", 3.2, 3),

        Student(5, "Антонов", 4.9, 4),

        Student(6, "Кузнецова", 4.1, 2),

        Student(7, "Абрамова", 4.3, 5),

        Student(8, "Иванов", 3.9, 3),

        Student(9, "Алексеев", 4.6, 4),

        Student(10, "Морозов", 3.5, 5)

    ]

    student_relations = [

        StudentClassRelation(1, 1),

        StudentClassRelation(1, 3),

        StudentClassRelation(2, 2),

        StudentClassRelation(3, 1),

        StudentClassRelation(3, 4),

        StudentClassRelation(4, 3),

        StudentClassRelation(5, 4),

        StudentClassRelation(6, 2),

        StudentClassRelation(7, 5),

        StudentClassRelation(8, 3),

```

```

StudentClassRelation(8, 1),
StudentClassRelation(9, 4),
StudentClassRelation(9, 5),
StudentClassRelation(10, 5),
]

processor = DataProcessor(classes, students, student_relations)

print("=" * 70)
print("ЗАДАНИЕ 1: Школьники с фамилией на 'А' и их классы")
print("=" * 70)
result1 = processor.get_students_starting_with_letter('А')
for last_name, class_name in result1:
    print(f"{last_name} - {class_name}")

print("\n" + "=" * 70)
print("ЗАДАНИЕ 2: Классы с минимальным средним баллом")
print("=" * 70)
result2 = processor.get_min_gpa_by_class()
for class_name, min_gpa in result2:
    print(f"{class_name}: {min_gpa} баллов")

print("\n" + "=" * 70)
print("ЗАДАНИЕ 3: Все связи школьник-класс")
print("=" * 70)
result3 = processor.get_all_student_class_relations()
for last_name, class_name in result3:
    print(f"{last_name} - {class_name}")

if __name__ == "__main__":
    main()

```

Результат выполнения программы:

=====

ЗАДАНИЕ 1: Школьники с фамилией на 'А' и их классы

=====

Андреев – 5А
Александрова – 5А
Антонов – 8А
Абрамова – 9Б
Алексеев – 8А

=====

ЗАДАНИЕ 2: Классы с минимальным средним баллом

=====

7В: 3.2 баллов
9Б: 3.5 баллов
6Б: 3.8 баллов
5А: 4.5 баллов
8А: 4.6 баллов

=====

ЗАДАНИЕ 3: Все связи школьник–класс

=====

Абрамова – 9Б
Александрова – 5А
Александрова – 8А
Алексеев – 8А
Алексеев – 9Б
Андреев – 5А
Андреев – 7В
Антонов – 8А
Иванов – 7В
Иванов – 5А
Кузнецова – 6Б
Морозов – 9Б
Петров – 6Б